

DecisionIQ Master Handbook

CHAPTER 11: Advanced Analytic Querying (Cohorts, RFM, and Hierarchies)

11.1 Cohort Retention Query

Tracks sign-up cohorts and calculates what percentage of them continue purchasing in subsequent months:

```
WITH CustomerCohorts AS (
    SELECT
        customer_id,
        STRFTIME('%Y-%m', acquisition_date) AS cohort_month
    FROM customers
),
CustomerActivity AS (
    SELECT DISTINCT
        customer_id,
        STRFTIME('%Y-%m', order_date) AS activity_month
    FROM orders
    WHERE status <> 'Cancelled'
),
CohortSizes AS (
    SELECT
        cohort_month,
        COUNT(customer_id) AS cohort_size
    FROM CustomerCohorts
    GROUP BY 1
),
CohortRetention AS (
    SELECT
        cc.cohort_month,
        ((CAST(SUBSTR(ca.activity_month, 1, 4) AS INTEGER) -
        CAST(SUBSTR(cc.cohort_month, 1, 4) AS INTEGER)) * 12) +
        (CAST(SUBSTR(ca.activity_month, 6, 2) AS INTEGER) -
        CAST(SUBSTR(cc.cohort_month, 6, 2) AS INTEGER)) AS month_number,
        COUNT(DISTINCT cc.customer_id) AS active_customers
    FROM CustomerCohorts cc
    JOIN CustomerActivity ca ON cc.customer_id = ca.customer_id
    GROUP BY 1, 2
)
SELECT
    cr.cohort_month,
    cs.cohort_size,
    cr.month_number,
    cr.active_customers,
    ROUND((cr.active_customers * 100.0) / cs.cohort_size, 2) AS retention_pct
FROM CohortRetention cr
```

```
JOIN CohortSizes cs ON cr.cohort_month = cs.cohort_month
ORDER BY cr.cohort_month DESC, cr.month_number ASC;
```

11.2 RFM Segmentation Query

Ranks customers into quintiles (1 to 5) for Recency, Frequency, and Monetary parameters using the `NTILE(5)` window function:

```
WITH CustomerMetrics AS (
  SELECT
    customer_id,
    (SELECT JULIANDAY(MAX(order_date)) FROM orders) -
    JULIANDAY(MAX(order_date)) AS recency_days,
    COUNT(order_id) AS frequency,
    SUM(total_amount) AS monetary_value
  FROM orders
  WHERE status <> 'Cancelled'
  GROUP BY customer_id
)
SELECT
  customer_id,
  NTILE(5) OVER (ORDER BY recency_days DESC) AS r_score,
  NTILE(5) OVER (ORDER BY frequency ASC) AS f_score,
  NTILE(5) OVER (ORDER BY monetary_value ASC) AS m_score
FROM CustomerMetrics;
```

11.3 Recursive Hierarchy Query

Finds the organizational manager reporting tree:

```
WITH RECURSIVE OrgHierarchy AS (
  SELECT
    employee_id,
    first_name || ' ' || last_name AS employee_name,
    role,
    reports_to,
    1 AS level_depth
  FROM employees
  WHERE reports_to IS NULL

  UNION ALL

  SELECT
    e.employee_id,
    e.first_name || ' ' || e.last_name AS employee_name,
    e.role,
    e.reports_to,
```

```
        h.level_depth + 1 AS level_depth
    FROM employees e
    JOIN OrgHierarchy h ON e.reports_to = h.employee_id
)
SELECT * FROM OrgHierarchy;
```